# ICT303 – Advanced Machine Learning and Artificial Intelligence

## Topic 4: Perceptron and Multilayer Perceptron (MLP) – Part II

Hamid Laga
H.Laga@murdoch.edu.au
Office: 245.1.020

[OFFICIAL]

# How to Get in Touch with the Teaching Team

- Internal and External Students
  - Email: H.Laga@murdoch.edu.au.
- Important
  - In any communication, please make sure that you
    - Start the subject of your email with ICT303
    - Include your student ID, name, and the lab slot in which you are enrolled.
  - We will do all our best to answer your queries within 24 hrs.


[OFFICIAL]

# In this Lecture

- Some important aspects to consider
  - Training neural networks (data, training, hyper parameters; see also Chapter 5.5 of the book)
  - Numerical stability and initialization (Chapter 5.4 of the book)
  - Dropout (Chapter 5.6 of the book)
- Interesting applications of MLP


[OFFICIAL]

# Training Neural Networks – Data Preparation

- Split the data set into three subsets: training, validation and testing
- Usually, we use
  - 70% of the data for training, 20% for validation and 10% for testing, or
  - 70% of the data for training, 10% for validation and 20% for testing


[OFFICIAL]

# Training Neural Networks – Data Preparation

- Training set
  - This is the data set you will use to learn the best parameters of the network
  - At each epoch,
    - update the network parameters and
    - measure the overall loss on this data set – this is called training loss.
- Validation set
  - Used only to decide when to stop training to avoid overfitting, and to tune the hyper parameters
  - DO NOT use it to update the network parameters
  - At each epoch,
    - measure the loss on this validation data set.
    - This is called generalization loss
- Testing set
  - Used to measure the actual performance of the trained network
  - It should never ever be used to learn the parameters or tune the hyper parameters


[OFFICIAL]

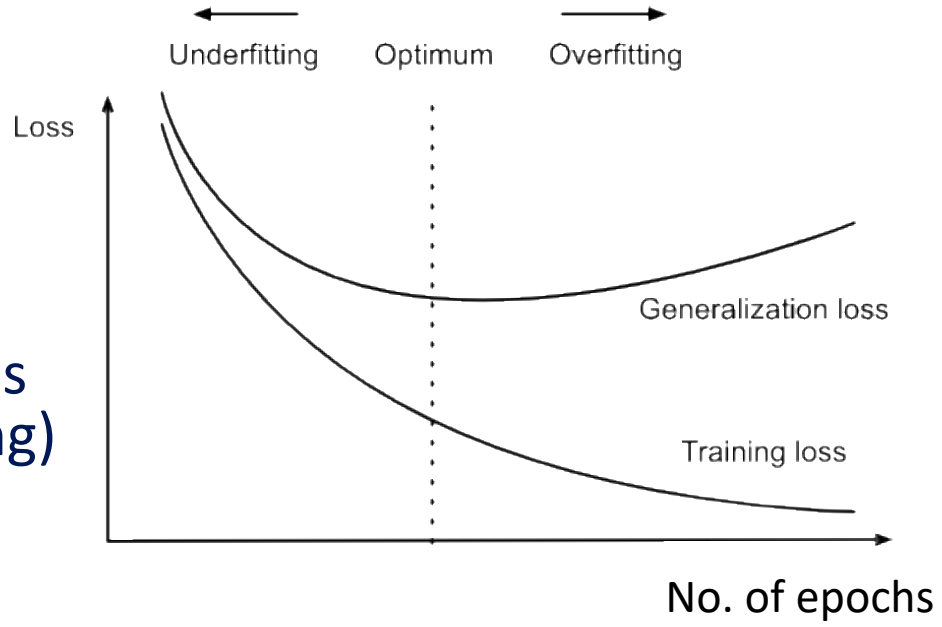
# Training Neural Networks – Training

- When should I stop training?
- Training for a very large number of iterations is not necessarily good
  - Instead of learning, the model will start memorizing the answers
  - It will then perform very poorly on an input that it did not see during training
  - This is called overfitting


[OFFICIAL]

# Training Neural Networks – Training

- When should I stop training?
- At each epoch
  - Training loss will decrease
  - Generalization loss will decrease, but at a certain point it starts increasing
- In the early epochs
  - The model is underfitting since both the training loss and generalization loss are high (although decreasing)
- When the generalization loss starts increasing, it means the model starts overfitting
  - You need to stop the training right before this point

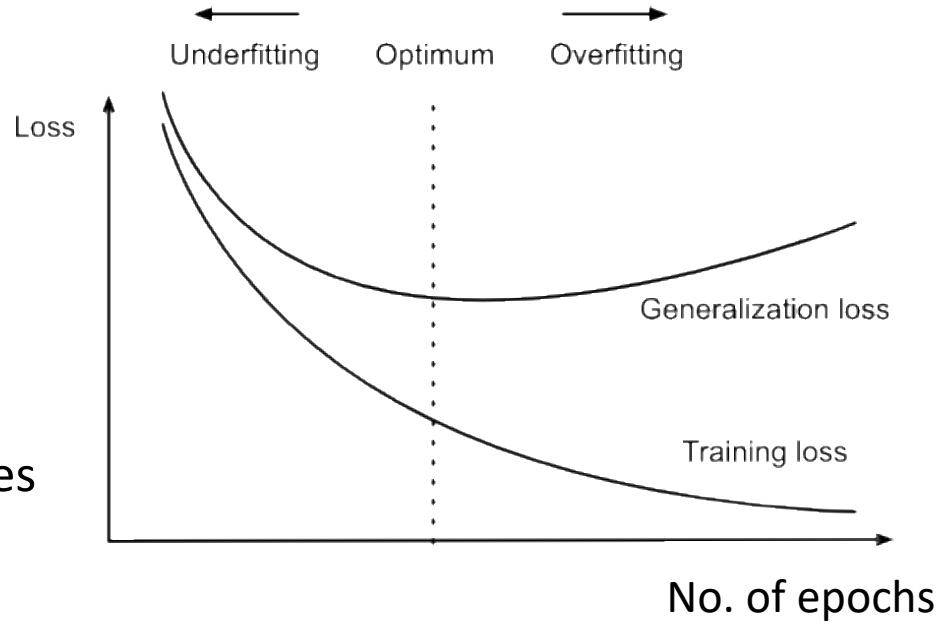




[OFFICIAL]

# Training Neural Networks – Training

- In your implementation, you need to
  - Compute the training and validation loss at each epoch
  - Save the state (the learned parameters) of the network at regular times, e.g., every 5 or 10 epochs
    - If the machine crashes, you only need to restart from the last saved state
  - Once training is finished,
    - Find the epoch no. where the generalization loss starts going up
    - The network parameters at that point are the optimal ones

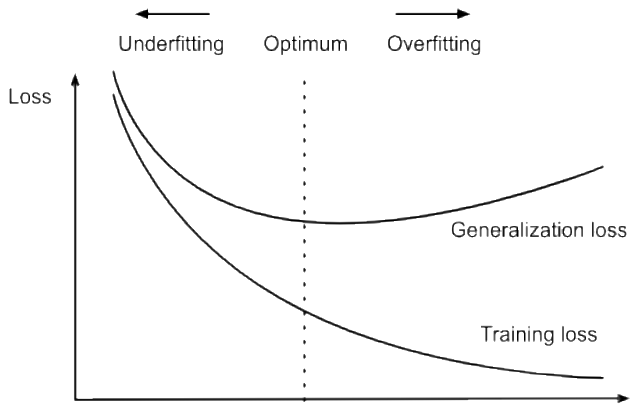




[OFFICIAL]

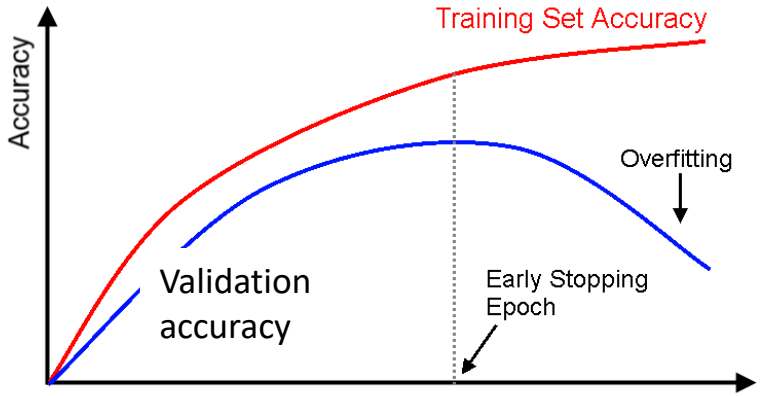
# Training Neural Networks – Training

- In addition to the loss, it is good to visualize the accuracy of the network on training and on validation
  - In classification, accuracy is the percentage of correct predictions obtained
  - The higher the accuracy is the better




No. of epochs

Loss curves




No. of epochs

Accuracy curves



[OFFICIAL]

# Training Neural Networks – Hyper Parameters

- A Deep Neural Network has many hyper parameters that can significantly affect performance
  - Architecture
    - Network type
    - Number of layers and number of neurons in each layer
    - Type of activation function at each layer
  - Learning
    - Learning rate
    - Optimizer type
    - Type and amount of training data
    - Batch size
- How to set them?


[OFFICIAL]

# Training Neural Networks – Hyper Parameters

- How to set them?
  - Test every possible configuration → impossible (a lot of time and expensive)
  - Use some intuitions to narrow down the list of options
    - Network type
      - Images → CNN, non-spatially structured data → MLPs
    - No. of layers and no. of neurons per layer
      - The more complex the problem the more layers you will likely need
      - Small training data set → small no. of layers and neurons per layer,
    - Activation functions → likely ReLU is the best (tested and validated by others)
    - Learning rate
      - Use scheduling: start with relatively large one and reduce it toa small one
    - Optimizers
      - Look at what others did, e.g., Adams is very popular
    - Batch size
      - Need to find a good trade-off between memory requirements, speed and accuracy (large batch size → faster, more memory but performance may drop at a certain point)


[OFFICIAL]

# Training Neural Networks – Hyper Parameters

- How to set them?
  - Test every possible configuration → impossible (a lot of time and expensive)
  - Use some intuitions to narrow down the list of options
    - Many tools have been developed to help automate/streamline this process
      - TensorBoard – runs for TensorFlow but can also be set for PyTorch
      - TensorBoardX – TensorBoard's extension to PyTorch
      - There are many other alternatives, see: https://neptune.ai/blog/the-best-tensorboard-alternatives


[OFFICIAL]

# In this Lecture

- Some important aspects to consider
  - Training neural networks (data, training, hyper parameters; see also Chapter 5.5 of the book)
  - Numerical stability and initialization (Chapter 5.4 of the book)
  - Dropout (Chapter 5.6 of the book)
- Interesting applications of MLP


[OFFICIAL]

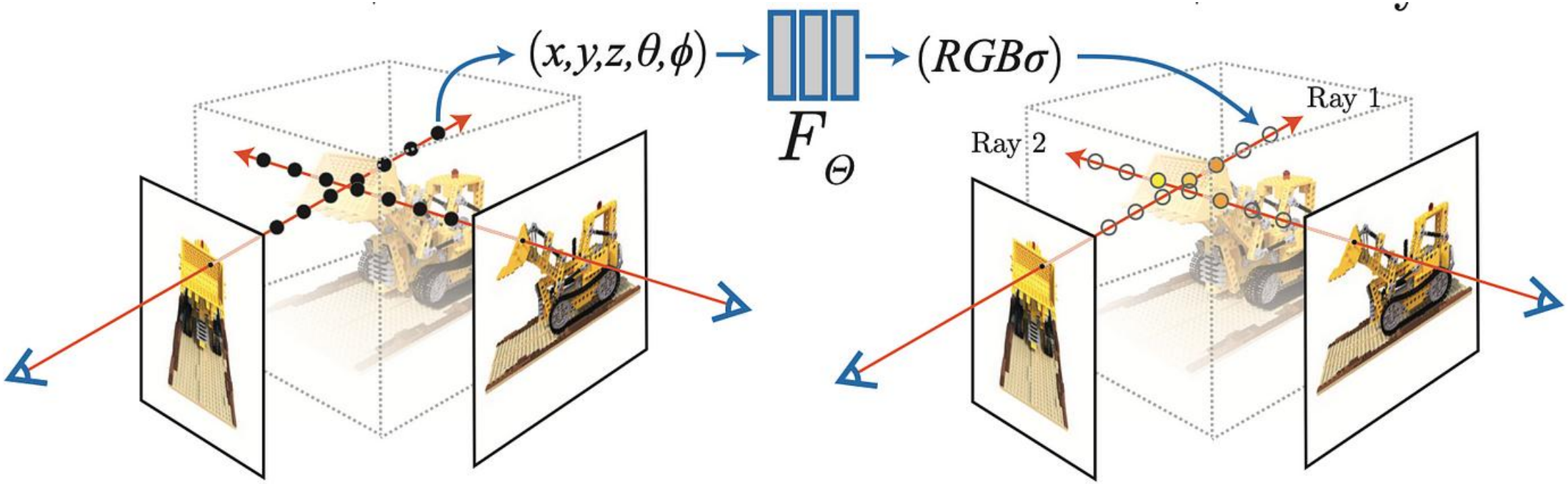
# Applications

- Novel view synthesis using Neural Radiance Fields
  - NeRF:
    - Official site https://www.matthewtancik.com/nerf
    - Others: https://www.youtube.com/watch?app=desktop&v=CRlN-cYFxTk
  - InstantNGP: https://nvlabs.github.io/instant-ngp/
  - NeuralAngelo: https://research.nvidia.com/labs/dir/neuralangelo/
  - 3D Gaussian Splatting:
    - Official site: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/
    - Other interesting videos
      - https://www.google.com/search?sca_esv=fd4a6f8cbbc02fb2&rlz=1C1GCHS_enAU1170AU1170&sxsrf=AE3TifMXK4W7zYARy8JDDYv53nhzqTfcvA:1755570903989&udm=7&fbs=AIIjpHxU7SXXniUZfeShr2fp4giZ1Y6MJ25_tmWITc7uy4KIeuYzzFkfneXafNx6OMdA4MQRJc_t_TQjwHYrzlkIauOKKs0YI23S9EeDMNq-iJuNE9kbk_tqcCBI4NkmSFsStjsRXP1Xs0hqcA8NrOIiNJKwjGLFWru1jr_OtncT3qc7s2sIkptL5fddmgm0rryFypvi7c4_TB9BquBpaSsdhJm_WiwfDA&q=3D+gaussian+splatting+in+paris+olympics&sa=X&ved=2ahUKEwi32vCg65WPAxWbTWwGHTMcFhMQtKgLegQIEhAB&biw=1920&bih=945&dpr=1#fpstate=ive&vld=cid:100e324f,vid:4MPb9JG9QW0,st:0



[OFFICIAL]

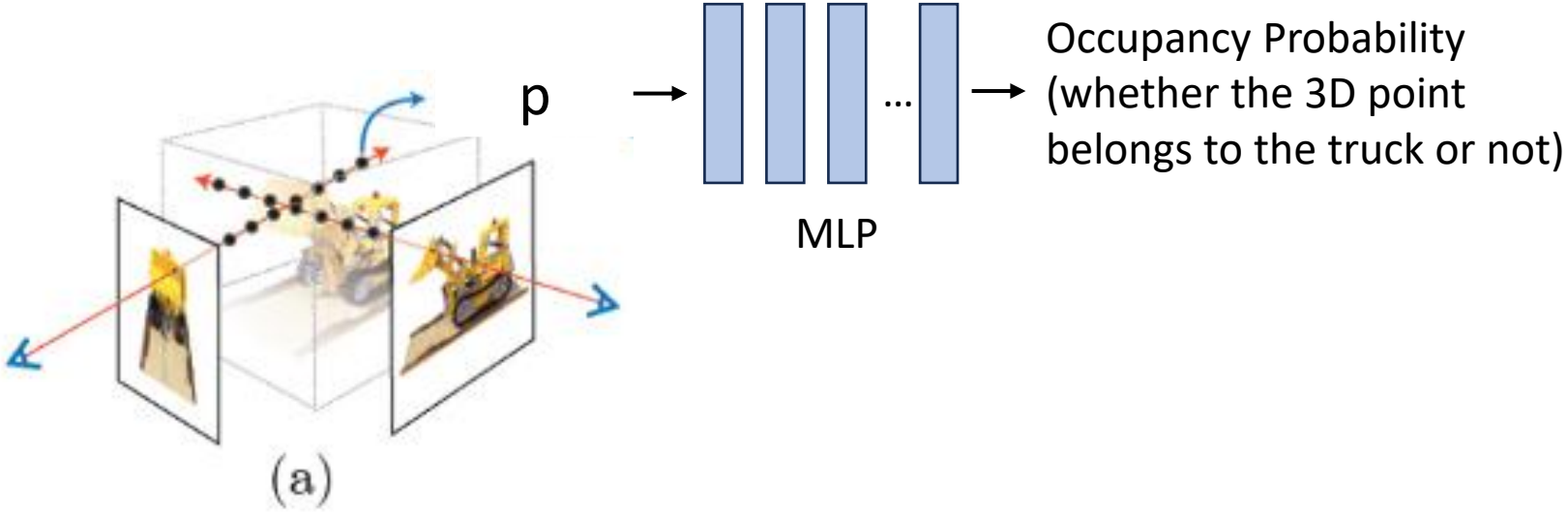
# Neural Radiance Fields




Neural Radiance Fields (NeRF)

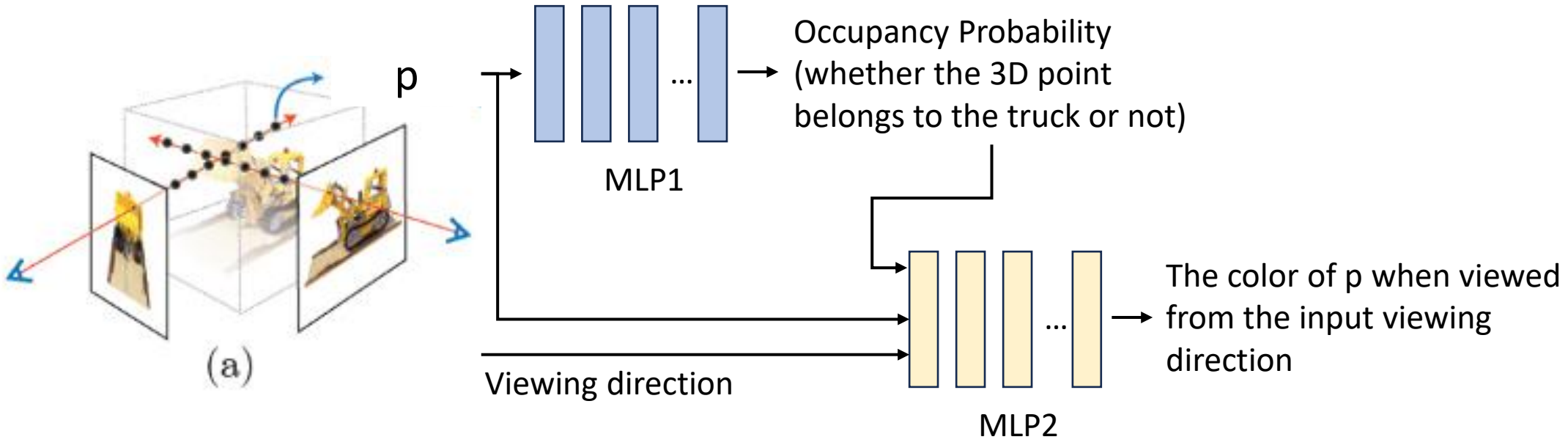
# Example 1 – 3D Reconstruction

- High level solution
  - We want a neural network that looks at the input images (or videos) and then decides for each point in the 3D space whether it belongs to the 3D object (Roger Federer's body) or not



# Example 1 – 3D Reconstruction + Rendering

- High level solution
  - We want a neural network that looks at the input images (or videos) and then decides for each point in the 3D space whether it belongs to the 3D object (Roger Federer's body) or not



# Applications

- 3D Reconstruction of RGB images of generic objects

[OFFICIAL]

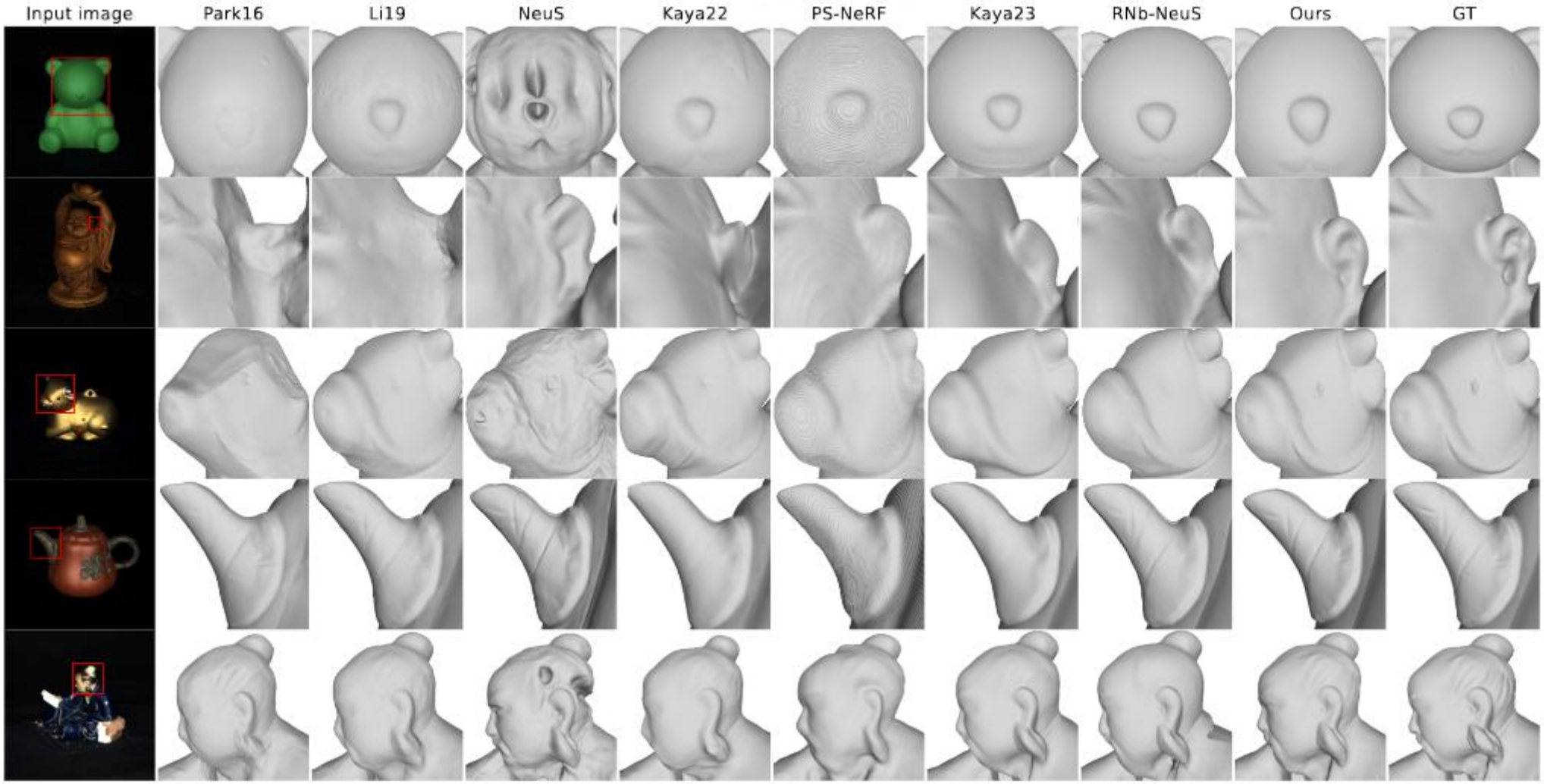
# Applications

- 3D Reconstruction of RGB images of generic objects

[OFFICIAL]

# Applications

- 3D Reconstruction of RGB images of generic objects



# Next Week

- Convolutional Neural Networks


[OFFICIAL]

# Questions


[OFFICIAL]